DIP LAB TASK:6

[Document subtitle]

AIR UNIVERSITY
AEROSPACE AND AVIATION CAMPUS
KAMRA

1. OBJECTIVE

The objective of this project is to design and develop a smart image enhancement system that improves the quality of low-resolution or low-contrast images. The system applies various digital image processing techniques including sampling, quantization, geometric transformations, intensity transformations, and histogram processing. The goal is to enhance visual quality and extract meaningful insights from the input image.

2. METHODOLOGY (PHASE-WISE EXPLANATION)

Phase 1: Image Acquisition & Understanding (Lab 01)

- The input image is loaded using Python (OpenCV).
- The RGB image is converted into grayscale for processing.
- Image properties are analyzed:
    - Resolution (height × width)
    - Data type (e.g., uint8)
    - Partial pixel matrix

Phase 2: Sampling & Quantization (Lab 02)

Sampling:

- Image is resized using different scales:
    - 0.25 (down-sampling)
    - 0.5
    - 1 (original)
    - 1.5
    - 2 (up-sampling)

Quantization:

- Bit depth is reduced:
    - 8-bit (original quality)
    - 4-bit (moderate quality loss)
    - 2-bit (high quality loss)

Phase 3: Geometric Transformations (Lab 03)

Applied transformations include:

- Rotation (30°, 45°, 60°, 90°, 120°, 150°, 180°)
- Translation (shifting image position)

- Shearing (distortion effect)

Phase 4: Intensity Transformations (Lab 04)

Applied transformations:

- Negative transformation (invert image)
- Log transformation (enhances dark regions)
- Gamma correction:
    - $\gamma = 0.5$ → brightens image
    - $\gamma = 1.5$ → darkens image

Phase 5: Histogram Processing (Lab 05)

- Histogram of original image is plotted.
- Pixel intensity distribution is analyzed.
- Histogram equalization is applied to improve contrast.

Phase 6: Final Integrated System

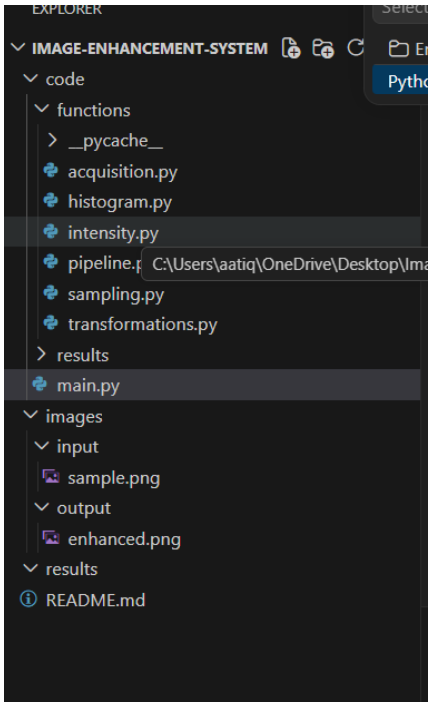
- A pipeline function is developed:

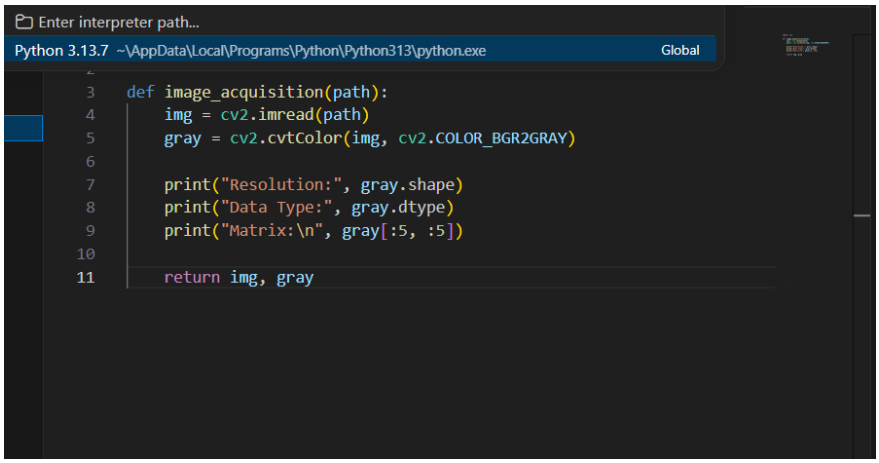
enhanced_image = process_image(input_image)

Steps:

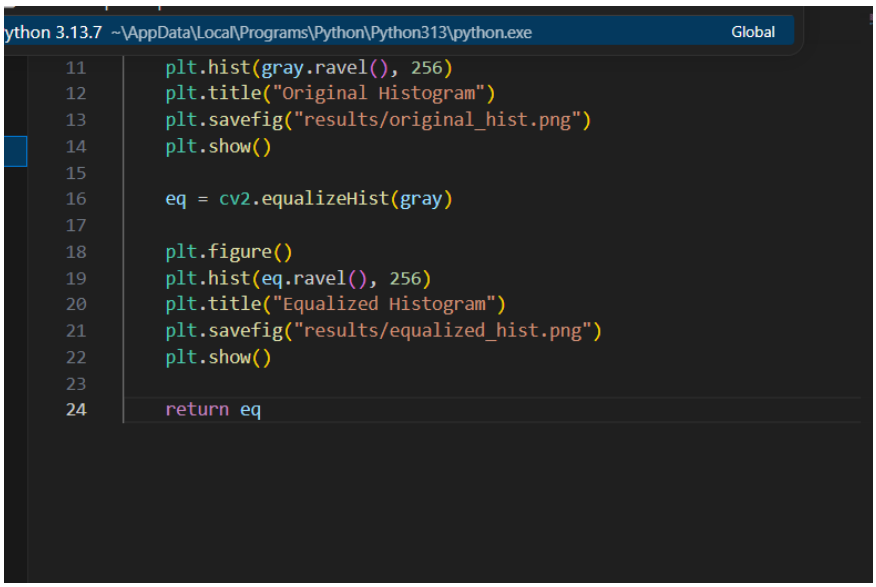
1. Convert to grayscale
2. Apply gamma correction
3. Apply histogram equalization

CODE SNIPPETS:






```python
def image_acquisition(path):
    img = cv2.imread(path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    print("Resolution:", gray.shape)
    print("Data Type:", gray.dtype)
    print("Matrix:\n", gray[:5, :5])

    return img, gray
```




```python
    plt.hist(gray.ravel(), 256)
    plt.title("Original Histogram")
    plt.savefig("results/original_hist.png")
    plt.show()

    eq = cv2.equalizeHist(gray)

    plt.figure()
    plt.hist(eq.ravel(), 256)
    plt.title("Equalized Histogram")
    plt.savefig("results/equalized_hist.png")
    plt.show()

    return eq
```

```python

    # SAFE LOG TRANSFORM
    log = 255 * np.log1p(gray) / np.log(256)
    log = np.uint8(np.clip(log, 0, 255))

    gamma05 = np.uint8(255 * (gray/255)**0.5)
    gamma15 = np.uint8(255 * (gray/255)**1.5)

    return {
        "negative": negative.astype(np.uint8),
        "log": log,
        "gamma05": gamma05,
        "gamma15": gamma15
    }
```

ME.md | intensity.py | pipeline.py | histogram.py | main.py

code > functions > pipeline.py > process_image

```python
import cv2
import numpy as np

def process_image(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gamma = np.uint8(255 * (gray/255)**0.5)
    enhanced = cv2.equalizeHist(gamma)
    return enhanced
```

code > functions > sampling.py > quantization

```python
import cv2
import numpy as np

def sampling(gray):
    scales = [0.25, 0.5, 1, 1.5, 2]
    results = {}

    for s in scales:
        resized = cv2.resize(gray, None, fx=s, fy=s)
        results[f"scale_{s}"] = resized

    return results

def quantization(gray):
    def quantize(img, bits):
        levels = 2**bits
        return (img // (256//levels)) * (256//levels)

    return {
        "8bit": quantize(gray, 8),
        "4bit": quantize(gray, 4),
        "2bit": quantize(gray, 2)
    }
```

code > functions > transformations.py > transformations

```python
import cv2
import numpy as np

def transformations(gray):
    rows, cols = gray.shape
    results = {}

    angles = [30,45,60,90,120,150,180]
    for a in angles:
        M = cv2.getRotationMatrix2D((cols/2, rows/2), a, 1)
        results[f"rotate_{a}"] = cv2.warpAffine(gray, M, (cols, rows))

    # Translation
    M = np.float32([[1,0,50],[0,1,30]])
    results["translated"] = cv2.warpAffine(gray, M, (cols, rows))

    # Shear
    M = np.float32([[1,0.5,0],[0,1,0]])
    results["sheared"] = cv2.warpAffine(gray, M, (cols, rows))

    return results
```

ode > main.py > ...

```python
def sampling():
        plt.imshow(resized, cmap='gray')
        plt.title(f"{s}x")
        plt.axis('off')
    plt.show()

# ---------------- QUANTIZATION ----------------
def quantization():
    def q(bits):
        levels = 2**bits
        return (gray // (256//levels)) * (256//levels)

    plt.figure("Quantization")
    for i, b in enumerate([8,4,2]):
        plt.subplot(1,3,i+1)
        plt.imshow(q(b), cmap='gray')
        plt.title(f"{b}-bit")
        plt.axis('off')
    plt.show()

# ---------------- TRANSFORMATIONS ----------------
def transformations():
    angles = [30,45,60,90,120,150,180]
    plt.figure("Transformations")
    for i, a in enumerate(angles):
        M = cv2.getRotationMatrix2D((gray.shape[1]/2, gray.shape[0]/2
        rot = cv2.warpAffine(gray, M, (gray.shape[1], gray.shape[0]))
        plt.subplot(3,3,i+1)
        plt.imshow(rot, cmap='gray')
        plt.title(f"{a}°")
        plt.axis('off')
```

intensity.py | transformations.py | histogram.py | main.py

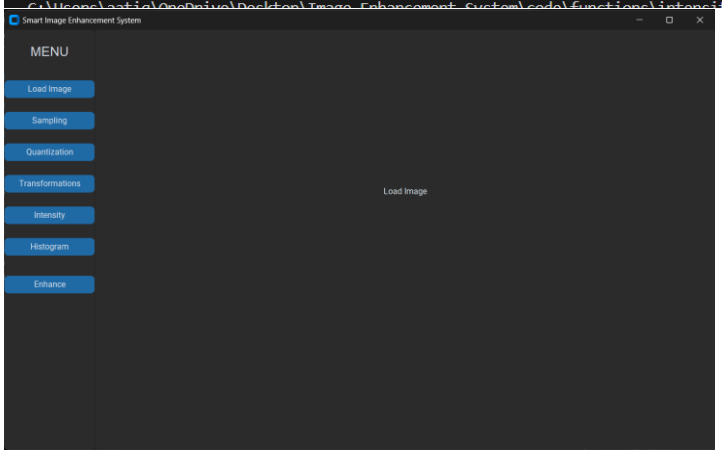
code > main.py > ...

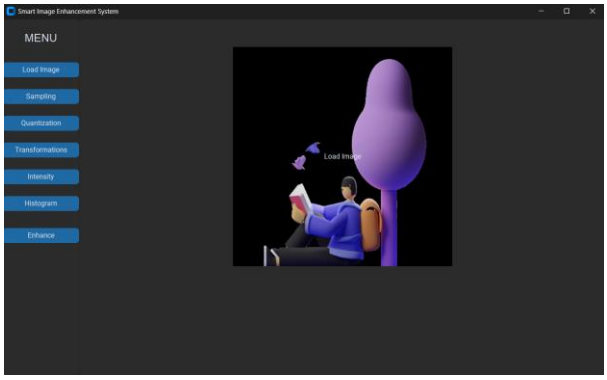
```python
    plt.show()

# ---------------- INTENSITY ----------------
def intensity():
    neg = 255 - gray
    log = np.uint8(255*np.log1p(gray)/np.log(256))
    g1 = np.uint8(255*(gray/255)**0.5)
    g2 = np.uint8(255*(gray/255)**1.5)

    imgs = [neg, log, g1, g2]
    titles = ["Negative","Log","Gamma 0.5","Gamma 1.5"]

    plt.figure("Intensity")
    for i in range(4):
        plt.subplot(2,2,i+1)
        plt.imshow(imgs[i], cmap='gray')
        plt.title(titles[i])
        plt.axis('off')
    plt.show()

# ---------------- HISTOGRAM ----------------
def histogram():
    eq = cv2.equalizeHist(gray)

    plt.figure("Histogram")
    plt.subplot(2,2,1), plt.imshow(gray, cmap='gray'), plt.title("Original")
    plt.subplot(2,2,2), plt.hist(gray.ravel(),256), plt.title("Hist")

    plt.subplot(2,2,3), plt.imshow(eq, cmap='gray'), plt.title("Equalized")
    plt.subplot(2,2,4), plt.hist(eq.ravel(),256), plt.title("Equalized Hist")
    plt.show()

# ---------------- PIPELINE ----------------
def enhance():
    gamma = np.uint8(255*(gray/255)**0.5)
    enhanced = cv2.equalizeHist(gamma)
    show_image(cv2.cvtColor(enhanced, cv2.COLOR_GRAY2BGR))
```

6.1

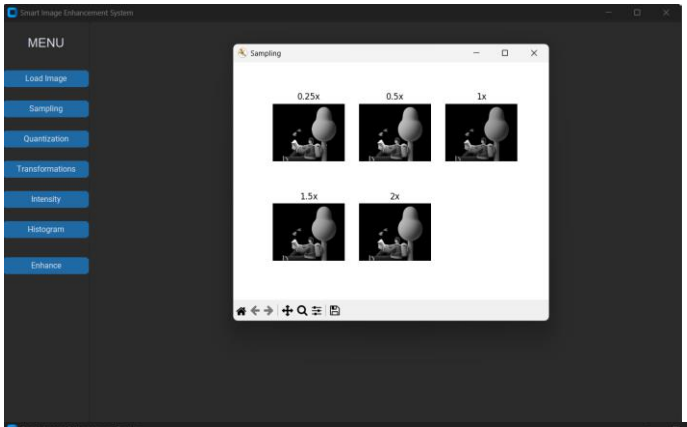
PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | python - code

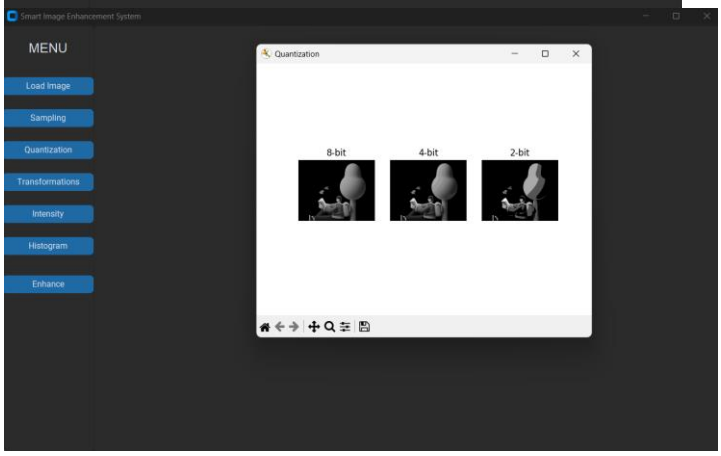
```
PS C:\Users\aatiq\OneDrive\Desktop\Image-Enhancement-System> cd code
PS C:\Users\aatiq\OneDrive\Desktop\Image-Enhancement-System\code> python main.py
Resolution: (512, 962)
Data Type: uint8
Matrix:
 [[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
```



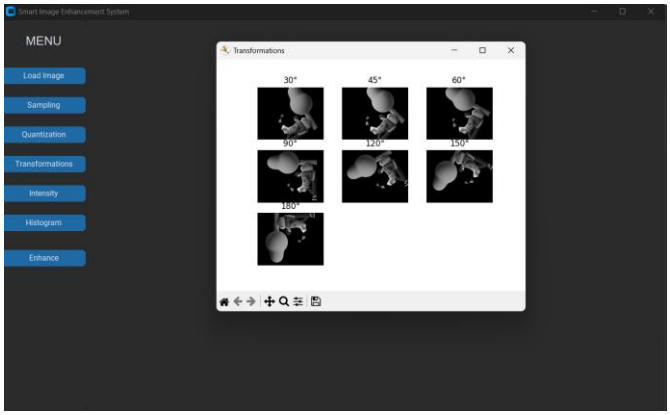

Smart Image Enhancement System
MENU
Load Image
Sampling
Quantization
Transformations
Intensity
Histogram
Enhance
Load Image

6.2

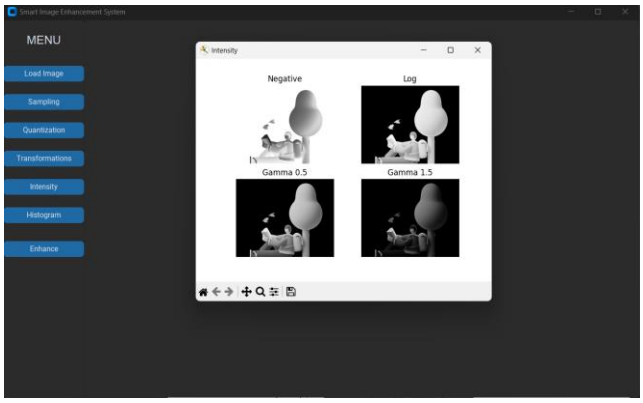

Smart Image Enhancement System
MENU
Load Image
Sampling
Quantization
Transformations
Intensity
Histogram
Enhance
Sampling
0.25x
0.5x
1x
1.5x
2x


Smart Image Enhancement System
MENU
Load Image
Sampling
Quantization
Transformations
Intensity
Histogram
Enhance
Quantization
8-bit
4-bit
2-bit

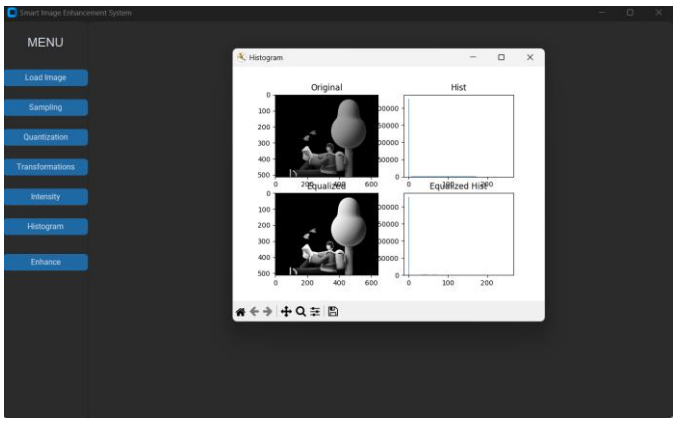
6.3


Smart Image Enhancement System
MENU
Load Image
Sampling
Quantization
Transformations
Intensity
Histogram
Enhance
Transformations
30°
45°
60°
90°
120°
150°
180°

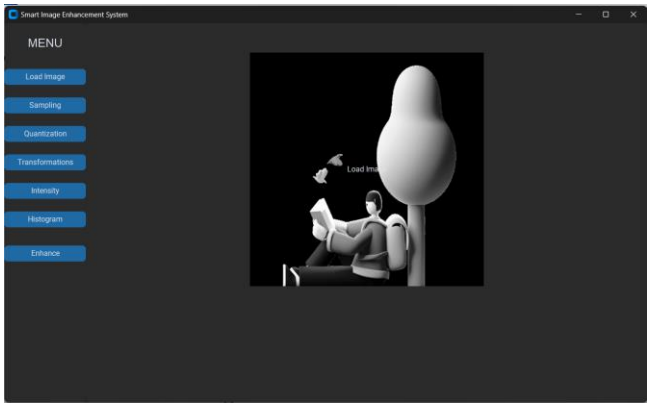
6.4



6.5



6.6



5. OBSERVATIONS & ANALYSIS

- Down-sampling reduces resolution and image clarity.
- Up-sampling increases size but may introduce blur.
- Quantization reduces colour depth and causes banding effects.
- Rotation and shearing alter spatial structure of the image.
- Negative transformation highlights edges.
- Log transformation enhances dark regions effectively.
- Gamma correction ($\gamma = 0.5$) is best for brightening.

- Histogram equalization significantly improves contrast in low-quality images.

6. FINAL CONCLUSION

The Smart Image Enhancement System successfully improves the visual quality of low-quality images using a structured processing pipeline. By combining intensity transformations and histogram equalization, the system enhances brightness, contrast, and overall clarity. The modular design allows flexibility and easy integration of different image processing techniques, making it suitable for real-world applications such as medical imaging, document processing, and social media enhancement.

QUESTION/ANSWERS:

1. Why does histogram equalization improve contrast?
Histogram equalization redistributes the intensity values of an image so that they span the full available range. This stretches areas with concentrated pixel values, making dark regions darker and bright regions brighter, which enhances overall contrast and reveals more details.
2. How does gamma affect brightness?
Gamma correction adjusts the brightness of an image non-linearly. A gamma value greater than 1 darkens the image, while a gamma less than 1 brightens it. It allows control over mid-tone intensities without drastically changing black and white levels.
3. What is the effect of quantization on image quality?
Quantization reduces the number of intensity levels in an image. Fewer levels can cause loss of detail, banding, or blockiness, especially in smooth gradients. More levels preserve finer details and produce smoother images.
4. Which transformation is reversible and why?
Linear transformations (like translation, scaling, rotation without rounding) are generally reversible because you can apply the inverse operation to recover the original image. Non-linear transformations, like logarithmic or gamma correction, may not be perfectly reversible due to rounding or information loss.
5. How do transformations affect spatial structure?
Geometric transformations (translation, rotation, scaling, shearing) alter the positions of pixels, changing the spatial layout of features. Point transformations (like contrast adjustment) modify pixel intensity without changing spatial arrangement. Improper transformations can distort or blur the spatial structure.